SMART CONTRACT AUDIT

ZOKYO.

May 6, 2021 | v. 1.0

PASS

Zokyo's Security Team has concluded that this smart contract passes security qualifications to be listed on digital asset exchanges.

TECHNICAL SUMMARY

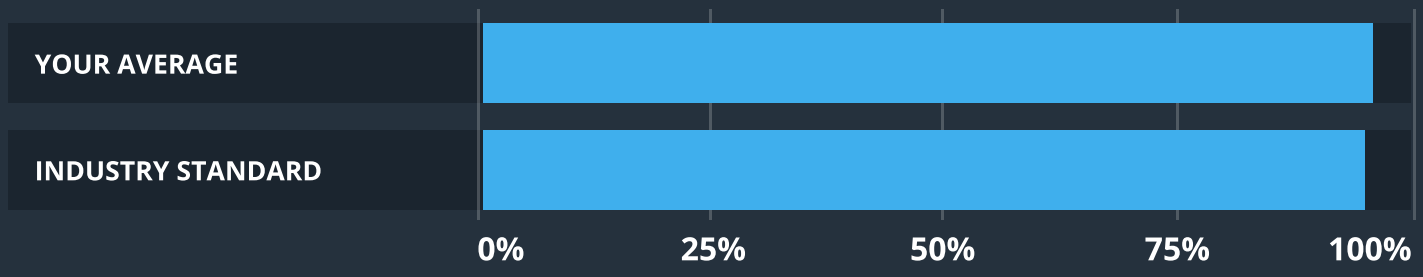
This document outlines the overall security of the Cross Pool smart contracts, evaluated by Zokyo's Blockchain Security team.

The scope of this audit was to analyze and document the Cross Pool smart contract codebase for quality, security, and correctness.

Contract Status

There was one critical issue found during the audit.

Testable Code



The testable code is 95.92%, which is above the industry standard of 95%.

It should be noted that this audit is not an endorsement of the reliability or effectiveness of the contract, rather limited to an assessment of the logic and implementation. In order to ensure a secure contract that's able to withstand the Ethereum network's fast-paced and rapidly changing environment, we at Zokyo recommend that the Hot Cross team put in place a bug bounty program to encourage further and active analysis of the smart contract.

TABLE OF CONTENTS

Auditing Strategy and Techniques Applied 3
Executive Summary 4
Structure and Organization of Document 5
Manual Review 6
Code Coverage and Test Results for all files 10
Tests written by Zokyo Secured team 10

AUDITING STRATEGY AND TECHNIQUES APPLIED

The Smart contract's source code was taken from the Cross Pool repository.

Throughout the review process, care was taken to ensure that the token contract:

- Implements and adheres to existing Token standards appropriately and effectively;
- Documentation and code comments match logic and behavior;
- Distributes tokens in a manner that matches calculations;
- Follows best practices in efficient use of gas, without unnecessary waste;
- Uses methods safe from reentrance attacks;
- Is not affected by the latest vulnerabilities;
- Whether the code meets best practices in code readability, etc.

Zokyo's Security Team has followed best practices and industry-standard techniques to verify the implementation of Cross Pool smart contracts. To do so, the code is reviewed line-by-line by our smart contract developers, documenting any issues as they are discovered. Part of this work includes writing a unit test suite using the Truffle testing framework. In summary, our strategies consist largely of manual collaboration between multiple team members at each stage of the review:

1	Due diligence in assessing the overall code quality of the codebase.	3	Testing contract logic against common and uncommon attack vectors.
2	Cross-comparison with other, similar smart contracts by industry leaders.	4	Thorough, manual review of the codebase, line-by-line.

EXECUTIVE SUMMARY

There was one critical issue found during the audit. All the mentioned findings may have an effect only in case of specific conditions performed by the contract owner.

Contracts are well written and structured. The findings during the audit have no impact on contract performance or security, so it is fully production-ready.

STRUCTURE AND ORGANIZATION OF DOCUMENT

For ease of navigation, sections are arranged from most critical to least critical. Issues are tagged “Resolved” or “Unresolved” depending on whether they have been fixed or addressed. Furthermore, the severity of each issue is written as assessed by the risk of exploitation or other unexpected or otherwise unsafe behavior:

Critical

The issue affects the ability of the contract to compile or operate in a significant way.

High

The issue affects the ability of the contract to compile or operate in a significant way.

Medium

The issue affects the ability of the contract to operate in a way that doesn’t significantly hinder its behavior.

Low

The issue has minimal impact on the contract’s ability to operate.

Informational

The issue has no impact on the contract’s ability to operate.

MANUAL REVIEW

CrossPool utilizes shared time lock & end investment block

CRITICAL | UNRESOLVED

CrossPool v1 specification points that **"Each pool will have an end block; i.e. the last block a reward will be distributed."**, but instead of that all pools are using end investment block & time lock params defined during CrossPool deployment.

Method massUpdatePools from CrossPool contract is vulnerable to `Out of gas`

HIGH | UNRESOLVED

Method massUpdatePools from CrossPool contract is iterating over all registered pools, and if the number of pools exceeds a specific number, then it will fail execution. It also affects the addition of new pools by using the method `add` and passing arguments withUpdate equal to true, when the list of pools already added is a greater specific number.

Snippet:

```
uint256 length = poolInfo.length;
for (uint256 pid = 0; pid < length; ++pid) {
  updatePool(pid);
}
```

CrossPool staker can get fewer reward tokens in case if RewardVault has less balance than accumulated reward tokens

HIGH | UNRESOLVED

RewardVault is not returning the actual amount of tokens transferred to Stacker and CrossPool contract marks that all reward tokens transferred.

Snippet 1:

```
if(block.number >= timeLock && totalPending > 0) {
 user.accClaim = 0;
 rewardVault.safeRewardTransfer(msg.sender, totalPending);
}
```

Snippet 2:

```
function safeRewardTransfer(
 address to,
 uint256 amount
) public onlyOwner {
 uint256 rewardTokenBalance = rewardToken.balanceOf(address(this));
 uint256 transferableAmount = amount > rewardTokenBalance
   ? rewardTokenBalance
   : amount;

 rewardToken.safeTransfer(to, transferableAmount);
}
```

Method initialize from CrossPool contract utilizes misleading names for arguments blockDuration & timeLockDuration

LOW | UNRESOLVED

Arguments blockDuration & timeLockDuration are used as a number of blocks instead of commonly used to represent a number of seconds.

Snippet:

```
startBlock = startBlock_;
endBlock = startBlock_.add(blockDuration);
timeLock = startBlock_.add(timeLockDuration);
```

RewardVault ownership has to be manually transferred to deployed CrossPool contract

INFORMATIONAL | UNRESOLVED

RewardVault contract ownership after deployment has to be manually or over automation, script be transferred to deployed CrossPool contract. Until that CrossPool contract is not operational.

RewardVault utilizes Proxy pattern and can be upgraded anytime after transferring ownership to CrossPool contract

INFORMATIONAL | UNRESOLVED

RewardVault is deployed utilizing the TransparentProxy pattern that utilizes ProxyAdmin contract address that is controlled by the initial contract deployer.

	Cross	RewardVault	CrossPool
Re-entrancy	Not affected	Not affected	Not affected
Access Management Hierarchy	Not affected	Not affected	Not affected
Arithmetic Over/Under Flows	Not affected	Not affected	Not affected
Unexpected Ether	Not affected	Not affected	Not affected
Delegatecall	Not affected	Not affected	Not affected
Default Public Visibility	Not affected	Not affected	Not affected
Hidden Malicious Code	Not affected	Not affected	Not affected
Entropy Illusion (Lack of Randomness)	Not affected	Not affected	Not affected
External Contract Referencing	Not affected	Not affected	Not affected
Short Address/ Parameter Attack	Not affected	Not affected	Not affected
Unchecked CALL Return Values	Not affected	Not affected	Not affected
Race Conditions / Front Running	Not affected	Not affected	Not affected
General Denial Of Service (DOS)	Not affected	Not affected	Not affected
Uninitialized Storage Pointers	Not affected	Not affected	Not affected
Floating Points and Precision	Not affected	Not affected	Not affected
Tx.Origin Authentication	Not affected	Not affected	Not affected
Signatures Replay	Not affected	Not affected	Not affected
Pool Asset Security (backdoors in the underlying ERC-20)	Not affected	Not affected	Not affected

CODE COVERAGE AND TEST RESULTS FOR ALL FILES

Tests written by Zokyo Secured team

As part of our work assisting Cross Pool in verifying the correctness of their contract code, our team was responsible for writing integration tests using the Truffle testing framework.

Tests were based on the functionality of the code, as well as a review of the Cross Pool contract requirements for details about issuance amounts and how the system handles these.

Contract: CrossPool

- ✓ cannot initialize twice (182ms)
- ✓ cannot initialize when timeLock exceed blockDuration (251ms)
- ✓ cannot initialize if rewardToken address is invalid (183ms)
- ✓ cannot initialize if reward vault address is invalid (246ms)
- ✓ should initialize correctly (661ms)
- ✓ cannot add staking token if address is invalid (387ms)
- ✓ cannot add staking token if caller is not owner (250ms)
- ✓ should add staking token correctly (707ms)
- ✓ should emit event on add (401ms)
- ✓ cannot set allocation points if caller is not owner (323ms)
- ✓ should set allocation points correctly (983ms)
- ✓ should emit event on set allocation points (669ms)
- ✓ cannot withdraw remaining rewards if caller is not owner (257ms)
- ✓ cannot withdraw remaining rewards after the start block (6140ms)
- ✓ withdraw remaining rewards should work correctly (960ms)
- ✓ should deposit correctly (2047ms)
- ✓ should deposit for multi users correctly (5576ms)
- ✓ cannot withdraw if amount is more than deposit amount (1345ms)
- ✓ should withdraw correctly (4505ms)
- ✓ should emit event on withdraw (1276ms)
- ✓ should withdraw emergency correctly (2128ms)
- ✓ should emit event on emergency withdraw (1809ms)
- ✓ should allow users to claim rewards accumulated before the timelock (10362ms)
- ✓ should depositing and withdraw correctly before the timelock (5703ms)
- ✓ should distribute rewards correctly (11544ms)

```
  ✓ should depositing and withdraw correctly after the end block (8798ms)
  ✓ should work correctly with multiple pools deposit (28293ms)

27 passing (3m)
```

FILE	% STMTS	% BRANCH	% FUNCS	% LINES	UNCOVERED LINES
contracts\	95.74	86.11	100.00	95.70	
CrossPool.sol	95.45	88.24	100.00	95.40	228, 229, 287, 288
RewardVault.sol	100.00	50.00	100.00	100.00	
contracts\libs\	100.00	100.00	100.00	100.00	
Misc.sol	100.00	100.00	100.00	100.00	
contracts\mocks\	100.00	100.00	100.00	100.00	
BEP20Mock.sol	100.00	100.00	100.00	100.00	
contracts\token\	100.00	100.00	100.00	100.00	
Cross.sol	100.00	100.00	100.00	100.00	
IBEP20.sol	100.00	100.00	100.00	100.00	
All files	**95.92**	**86.11**	**100.00**	**95.92**	

We are grateful to have been given the opportunity to work with the Hot Cross team.

The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them.

Zokyo's Security Team recommends that the Hot Cross team put in place a bug bounty program to encourage further analysis of the smart contract by third parties.

ZOKYO.